Swing 1 (¡Hola mundo!)

Vamos a empezar a trabajar con interfaces gráficas con Java Swing creando una ventana (JFrame) con las siguientes características:

- Título "¡Hola mundo!".
- Tamaño de 300 píxeles de ancho y 200 píxeles de alto.

Ayuda: Para crear la ventana, crea una clase que herede de JFrame. Incorpora al constructor de la clase el código para definir el título de la ventana y su tamaño. Luego define el método main() en el que se crea una instancia de la nueva clase y se hace visible. Revisa la documentación de Java de la clase JFrame para resolver este ejercicio.

Experimenta con más propiedades de la ventana consultando la ayuda del API:

- Modifica el color de fondo de la ventana y ponlo en color RGB: 143, 201, 58.
- Evita que se pueda cambiar de tamaño de la ventana.
- Añade comportamiento al icono de cerrar la ventana (por defecto no hace nada).
- Ubica la ventana centrada en la pantalla.

Swing 2 (Layouts)

El objetivo de esta práctica es familiarizarse con el funcionamiento de los principales modelos de distribución de componentes en contenedores: BorderLayout, GridLayout, FlowLayout y BoxLayout. Usando como referencia el ejemplo proporcionado, debes crear el diseño de una calculadora básica con la distribución que se muestra en la imagen. Para crear la calculadora, debes tener en cuenta lo siguiente:

- Dentro del panel principal, el JTextField que representa el display no es editable, el texto se alinea a la derecha y ocupa la parte superior del panel principal de la ventana (BorderLayout.NORTH). Intenta modificar la tipografía y tamaño de letra del JTextField usando el método setFont() para que la fuente sea Courier, con decoración Font.BOLD y su tamaño 24 puntos.
- Los botones se distribuyen en una cuadrícula de 4 filas x 4 columnas con una separación vertical y horizontal de 10 píxeles. Los botones ocupan el área central (BorderLayout.CENTER) del panel principal de la ventana. Además ocupan un espació mucho mayor que el display.
- Una vez creado el diseño principal, *¿cómo modificarías el color del texto o el fondo de los botones?*



Swing 3 (Eventos)

Esta práctica tiene por objeto familiarizarse con el manejo de eventos. Para ello, partiendo del código proporcionado y la solución a la práctica Swing 2 (Layouts) debes completar la funcionalidad de la Calculadora. Para resolver esta práctica debes utilizar únicamente la interface ActionListener y la clase ActionEvent.

Resuelve la práctica de manera incremental:

- En primer lugar, intenta que se añadan dígitos, operadores o el punto decimal al display.
- Después, controla que no se pueden poner dos operadores o dos puntos decimales seguidos. Añade mensaje de error (usando la clase JOptionPane) cada vez que se pulsa un botón que no corresponde.
- Para el caso del botón "=", no te preocupes en calcular las operaciones (que es una tarea compleja). Controla únicamente que sólo se puede poner un "=" después de un dígito.

Swing 4 (JTable)

Esta práctica tiene por objeto familiarizarse con el renderizado y edición en JTable. Se toma como referencia el código de un ejemplo sencillo en la que hay una interfaz con dos tablas que usan el renderizado y edición 'por defecto': JLabel con el valor toString() de cada atributo y JTextField. A partir de esta interfaz debes realizar las siguientes modificaciones:

Modificaciones en la tabla de Comics:

- Renderizar la editorial como una imagen con el logo de DC o MARVEL (usa los logos de la carpeta 'images') y añadir el nombre de la editorial como ToolTipText.
    - TableCellRenderer, JLabel.setIcon() & JComponent.setTooltipText()
- Modificar la altura de todas las filas de la tabla a 26 píxeles para que se vea mejor la imagen.
    - JTable.setRowHeight()
- Renderizar las filas pares e impares con un color de fondo específico: RGB(250, 249, 249) y RGB(190, 227, 219) respectivamente.
    - JComponent.setBackground()
- Renderizar las celdas seleccionadas con el color de renderizado por defecto.
    - JComponent.setBackground(JTable.getSelectionBackground())
- Alinear las columnas de texto a la izquierda y las columnas numéricas al centro. Debes alinear tanto la cabecera como los datos de la tabla.
    - TableCellRenderer & JTable.getTableHeader().setDefaultRenderer()
- Impedir reordenar las columnas de la tabla.
    - JTable.getTableHeader().setReorderingAllowed(boolean)
- Redimensionar el tamaño de la columna título a 400 píxeles.
    - JTable.getColumnModel().getColumn().setPreferredWidth()
- Editar la editorial a partir de un JComboBox en el que se puede seleccionar el valor de la Editorial en texto.
    - new DefaultCellEditor(JComboBox)
- Intenta editar algún valor numérico de la tabla haciendo doble click. Después de editar, ¿cómo se alinéan los valores numéricos? ¿qué puedes hacer para corregirlo?
    - Integer.parseInt(value.toString())

Modificaciones en la tabla de Personajes:

- Renderizar la editorial como una imagen con el logo de DC o MARVEL (usa los logos de la carpeta 'images') y añadir el nombre de la editorial como ToolTipText.

- Modificar la altura de todas las filas de la tabla a 26 píxeles para que se vea mejor la imagen.
- Renderizar las celdas seleccionadas con el color de renderizado por defecto.
- Alinear las columnas de texto a la izquierda y las columnas numéricas al centro. Debes alinear tanto la cabecera como los datos de la tabla.
- Redimensionar el tamaño de las columnas nombre y email a 200 píxeles.
- Editar la editorial a partir de un JComboBox en el que se puede seleccionar el valor de la Editorial en texto.
- ¿El color de fondo de las celdas cuando la tabla está vacía y cuando se cargan personajes es el mismo? Intenta usar el color inicial de la tabla.
    - result.setBackground(table.getBackground())
    - result.setForeground(table.getForeground())

Swing 5 (Eventos de teclado y ratón)

Esta práctica toma como base la solución parcial de la práctica 4 y su objetivo es el trabajo con eventos de todo tipo. La nueva GUI de la ventana principal incorpora una caja de texto para filtrar los cómics por título. Tomando como referencia esta nueva versión del ejemplo, realiza las siguientes modificaciones. Además, se proporciona el esqueleto de un cuadro de diálogo para modificar los personajes de un cómic. Las modificaciones que debes realizar para este proyecto son las siguientes:

- Bloquea la edición de las columnas ID y Personajes en la tabla de Comics para que no puedan modificarse.
    - Redefinir el método public boolean isCellEditable(int row, int column) al inicializar la tabla.
- Bloquea la edición de todas las columnas en la tabla de Personajes para que no pueda modificarse ningún dato.
    - Redefinir el método public boolean isCellEditable(int row, int column) al inicializar la tabla.
- Filtra la tabla de cómics a partir del valor que se introduzca en la caja de texto. A medida que se va introduciendo texto, los comics que aparecen en la tabla deben incluir el texto del filtro en su título. Si no hay texto, aparecerán todos los comics.
    - Implementar un DocumentListener y añadirlo al JTextfield.getDocument().addDocumentListener()
    - ¿Podrías resaltar el texto del filtro con un color? (busca una solución en la documentación de JLabel en el Java Tutorial)

        - Añadir al texto del JLabel formato HTML: <html>texto</html>
- Marca con el color de selección por defecto de la tabla de personajes la fila de la tabla de personajes sobre la que se encuentra el ratón. Para ello, detecta los movimientos del ratón y modifica el renderizado de toda la fila sobre la que se posicione el ratón.

    - Crear una variable global entera para guardar el número de fila de la tabla sobre la que está el ratón (se inicializa con -1).
    - Crear un MoseMotionAdapter y redefinir el método mouseMoved() para transformar la posición del ratón en un número de fila usando el método JTabel.rowAtPoint().
    - Crear un MouseAdapter y redefinir el método mouseExited() para detectar cuándo el ratón sale de la tabla para actualizar la variable global a -1.
    - Añadir los dos listeners/adapters a la tabla de personajes.

- Actualizar el TableCellRenderer para modificar el color de la celda cuando la fila sea igual al valor de la variable global. Puedes usar los colores por defecto de las celdas seleccionadas para el fondo y el texto.
- Cada vez que se actualiza la posición hay que forzar el redibujado de la tabla con el método repaint() para que el TableCellRenderer pinte la celda de manera diferente.

Añade nuevos comics

- Al pulsar la combinación de teclas CTRL + C, abre un cuadro de diálogo para añadir un nuevo cómic. El cuadro de diálogo pedirá únicamente la editorial (desde un JComboBox con las imágenes de las editoriales) y el título. El valor del ID se coge a partir del ID más alto. El nuevo cómic no tendrá personajes.
  - Crear un KeyListener y redefinir el método keyPressed() para detectar la pulsación de la tecla C y CTRL (e.getKeyCode() == KeyEvent.VK_C && e.isControlDown()).
- Para abrir este cuadro de diálogo usa la clase JOptionPane, en concreto debes usar el método estático showConfirmDialog().
  - Para personalizar los campos del cuadro de diálogo se crea un array JComponent[] con el JComboBox (inicializado previamente) para la editorial, el JTextField para el título y dos JLabel. Este array de componentes se pasa como segundo parámetro del método showConfirmDialog(). Los componentes se distribuyen en el cuadro de diálogo desde la esquina superior izquierda y van progresando hacia la derecha fila a fila.
  - Para renderizar los elementos del JComboBox como una imagen se utiliza un ListCellRenderer que es muy parecido al TableCellRenderer. El método setRenderer() permite asignar el renderer personalizado al JComboBox.
- Una vez creado el nuevo cómic, debes añadirlo a la lista de comics y actualizar la tabla de comics para que aparezca el nuevo cómic.
  - El método showConfirmDialog() devuelve un valor entero cuando se cierra el cuadro de diálogo. Ese valor numérico representa la opción pulsada, por ejemplo JOptionPane.OK_OPTION representa que se ha pulsado el botón OK.
  - Cuanto se confirma que se ha pulsado el botón OK, se crea un nuevo cómic a partir de los valores de la editorial seleccionada en el JComboBox y el título del JTextField. El nuevo comic creado se añade a la lista de comics y para visualizarlo en la tabla es necesario cargar de nuevo los comics con el método loadComics().
  - Al intentar añadir el nuevo cómic a la lista de comics se lanza una excepción, ¿a qué se debe este error?
    - La lista que se crea en el método main() es ¡***inmutable***!. Es necesario crear un ArrayList<Comic> que permita añadir elementos.

Modifica los personajes asociados a un cómic

- Si está seleccionado un cómic en la tabla de cómics y se pulsa la combinación de teclas CTRL + P se abre un cuadro de diálogo para añadir personajes al cómic.
    - Crear un KeyListener y redefinir el método keyPressed() como en la modificación anterior pero ahora además, hay que comprobar que hay una fila seleccionada en la tabla de comics (tablaComics.getSelectedRow() != -1).
- El esqueleto del cuadro de diálogo ya está creado (JDialogPersonajes). Su estructura es la siguiente, a la derecha contiene un JList con los personajes del cómic seleccionado, a la izquierda otro JList con los personajes de todos los cómics, y en el centro botones para mover personajes de un JList al otro.
    - El cuadro de diálogo recibe dos parámetros en el constructor: la lista de personajes de todos los cómics eliminando los personajes del cómic seleccionado y el cómic seleccionado. El primer parámetro implica crear un List<Personaje> a partir de los personajes de todos los cómics, omitiendo los personajes del cómic actual (se recorren todos los comics y todos los personajes de cada cómic omitiendo los valores repetidos y los personajes del cómic seleccionado).
- Al pulsar los botones debes realizar los movimientos oportunos de personajes entre los JList y actualizar de manera correcta la lista de personajes del cómic seleccionado. El cuadro de diálogo se cierra desde la "X" de cerrar la ventana. Una vez cerrado el cuadro de diálogo debes actualizar la tabla de comics para que quede reflejado el nuevo número de personajes.
    - Para mover personajes de un JList a otro hay que implementar un ActionListener en cada botón. El método getSelectedValue() devuelve el personaje seleccionado en una lista y los métodos removeElement() y addElement() de cada uno de los DefaultListModel permiten eliminar y añadir elementos a cada JList.
    - Además de actualizar los JList, hay que modificar la lista de personajes del cómic actual (añadiendo o eliminando el personaje seleccionado).
    - Una vez cerrado el cuadro de diálogo hay que cargar de nuevo las tablas para que los cambios se vean reflejados. Para hacer esto es necesario bloquear la interacción con el JFrame principal. Esto puede hacerse modificando la modalidad del JDialog mediante los métodos setModal() o setModalityType().

Swing 6 (JTree)

El objetivo de esta práctica es realizar un ejercicio de tipo examen. Usando como referencia el examen de la convocatoria ordinaria de enero de 2022 el objetivo de esta práctica es el uso del componente JTree y el uso de un render personalizado. Se proporciona el código base para leer una lista de lanzamientos desde una base de datos y se pide lo siguiente:

- A partir de la lista de lanzamientos, crear un mapa que indexe los lanzamientos por compañía.
- Añadir la funcionalidad a la ventana que se proporciona para usar un JTree que visualice los lanzamientos agrupados por compañía.
    - El nodo raíz del árbol mantiene el icono por defecto y muestra el número total de lanzamientos.
    - El segundo nivel del árbol representa a cada compañía: el icono es el logo de la compañía (ver archivos de la carpeta images) escalado a 20x20 píxeles.
    - El tercer nivel (nodo hoja) representa cada lanzamiento: se usa el icono por defecto y el texto mostrado incluye la fecha del lanzamiento en formato dd-MMM-yyyy HH:mm y la localización del lanzamiento. Además, se usa como color de texto (verde o rojo) dependiendo del estado del lanzamiento (SUCCESS o FAIL).

Ayuda para la resolución:

- Puedes tomar como referencia la ayuda específica de JTree del Java Tutorial.
- Para la creación de nodos revisa la clase DefaultMutableTreeNode
- Para el renderizado personalizado revisa la clase DefaultTreeCellRenderer